sv-tpu-core

UVM Verification Test Plan

Parameterized, Weight-Stationary Systolic Array MMU

Verification Lead: Atharva Kulkarni

Document scope: test cases, mutation-based assertion validation, SVA reference, and functional coverage plan.

Base Test - 5

Table of Contents

Table of Contents 2
1. Overview 6
1.1 Test Categories at a Glance 6
2. Test Case Index 6
3. Detailed Test Cases 8
3.1 Category 1 — Basic Functional Correctness 8
TC-001 — Full 4×4 Random Matrix Multiply 8
TC-002 — 3×3 Subarray 8
TC-003 — 2×2 Subarray 8
TC-004 — 1×1 Scalar MAC 9
TC-005 — All-Zero Activation Matrix 9
TC-006 — All-Zero Weight Matrix 9
TC-007 — Max int8 Values (Worst-Case Accumulation) 9
TC-008 — Min int8 Values (Negative Accumulation) 10
TC-009 — Mixed Positive and Negative Values 10
TC-010 — Identity Matrix as Weights 10
3.2 Category 2 — Back-to-Back Transaction Sequencing 10
TC-011 — 10+ Consecutive 4×4 Computations, No Gap 11
TC-012 — Back-to-Back With One-Cycle Gap 11
TC-013 — Back-to-Back With Alternating Dimensions 11
3.3 Category 3 — Weight Poison & Reset Stress 12
TC-014 — Weight Poison: All-Zero Weights 12
TC-015 — Weight Poison: All-127 Weights 12
TC-016 — Weight Poison: Identity Matrix Weights 12
TC-017 — Weight Poison: Checkerboard Pattern 13
TC-018 — Reset Stress: Reset During WEIGHT_LOAD 13
TC-019 — Reset Stress: Reset During PE_CLEAR 13
TC-020 — Reset Stress: Reset During ACTIVATION_FLOW 14
3.4 Category 4 — Reset Behavior 14
TC-021 — Synchronous Reset From IDLE 14
TC-022 — Reset Then Immediate Restart 15
TC-034 — Reset Asserted While FSM Is in DONE State 15
3.5 Category 5 — Illegal Operation / Error Injection 16
TC-023 — Write to Read-Only STATUS_REG 16
TC-024 — START Asserted Before DIM_REG Written (dim=0) 16

Virt. Seq.

TC-025 — Double START (Second START Mid-Computation) 16
TC-026 — Invalid Dimension (DIM_REG > 4) 17
3.6 Category 6 — Latency & Throughput Performance 17
TC-027 — Single N=1 Latency Verification 17
TC-028 — Single N=2 Latency Verification 17
TC-029 — Single N=3 Latency Verification 18
TC-030 — Single N=4 Latency Verification (Worst Case) 18
TC-031 — Back-to-Back Throughput at N=4 18
3.7 Register Abstraction Layer (RAL) Built-In Sequences 19
TC-032 — RAL Built-In: uvm_reg_hw_reset_seq 19
TC-033 — RAL Built-In: uvm_reg_access_seq 19
4. Negative Testing — Assertion Validation via Force Injection 21
TC-035a 21
TC-035b 21
TC-035c 21
TC-035d 21
TC-035e 21
TC-035f 22
TC-035g 22
5. SVA Assertion Reference 23
A1 — axi_awvalid_stable 23
A2 — axi_wvalid_with_awvalid 23
B1 — no_skip_weight_load 23
B2 — pe_clear_one_cycle 24
B3 — result_latency 24
B4 — no_spurious_done 24
C1 — zero_input_no_accumulate 25
D1 — signal_level_latency 25
D2 — back_to_back_throughput 25
6. Functional Coverage Plan 27
cp_dim — Matrix Dimension 27
cp_weight_pattern — Weight Matrix Character 27
cp_activation_pattern — Activation Matrix Character 27
cx_dim_x_weight — Cross: Dimension × Weight Pattern 28
cx_dim_x_act — Cross: Dimension × Activation Pattern 28
cp_back_to_back — Inter-Transaction Gap 28

cp_error_type — Error Injection Scenario 28
cp_reset_state — FSM State at Reset 29
7. Scoreboard Architecture & Pass Criteria 30
7.1 Path 1 — Positive Checking (Normal Functional Tests) 30
What It's Doing 30
How the Scoreboard Knows to Use This Path 30
The Checking Procedure — Step by Step 30
Pass Criterion 31
What Gets Logged on a Mismatch 31
7.2 Path 2 — Negative Checking (Error Injection Tests) 31
What It's Doing 31
The Checking Procedure — Step by Step 31
Pass Criterion 32
7.3 Path 3 — Performance Checking (Latency and Throughput) 32
What It's Doing 32
Two Checkers, One Contract 32
Throughput Check (TC-031) 33
Pass Criterion 33
7.4 Path 4 — RAL Checking (Register Model Correctness) 33
What It's Doing 33
How the RAL Model Checks Automatically 33
What the Scoreboard Adds on Top 33
Pass Criterion 34
7.5 Summary — The Four Pass Criteria 34
7.6 Global Rule Across All Paths 34
8. Assertion Exercise Mapping 35
8.1 File 1 — axi_lite_sva.sv (bound to axi_lite_slave.sv) 35
A1 — axi_awvalid_stable 35
A2 — axi_wvalid_with_awvalid 35
8.2 File 2 — mmu_controller_sva.sv (bound to mmu_controller.sv) 36
B1 — no_skip_weight_load 36
B2 — pe_clear_one_cycle 36
B3 — result_latency 37
B4 — no_spurious_done 37
8.3 File 3 — pe_sva.sv (bound to pe.sv, all N×N instances) 38
C1 — zero_input_no_accumulate 38

8.4 File 4 — mmu_perf_checker.sv (bound to mmu_top.sv) 39
D1 — signal_level_latency 39
D2 — back_to_back_throughput 39
8.5 TC-035 — Directed Forced Violation Test Summary 40
8.6 Summary — All Nine Assertions 40

1. Overview

This document defines the verification test plan for sv-tpu-core, a parameterized, weight-stationary systolic array modeled after the Google TPU v1 matrix-multiply unit (MMU). It enumerates 34 functional and structural test cases spanning seven categories, 7 mutation-based negative tests used to validate the assertion suite itself, a reference for all 9 SystemVerilog Assertions (SVA) bound into the design, and the 8 functional coverage groups used to measure verification closure.

Each test case specifies its stimulus, expected behavior, scoreboard check, the assertions it is expected to exercise, and the functional coverage bins it targets. Where a test case is paired with a specific JasperGold formal proof or carries phase-ordering significance, that context is called out explicitly.

1.1 Test Categories at a Glance

Category	Focus
Category 1 — Basic Functional Correctness (10)	Validates core matrix-multiply correctness across array dimensions, value extremes, and weight/activation patterns, checked against a Python golden reference model.
Category 2 — Back-to-Back Transaction Sequencing (3)	Stresses consecutive computations with no gap, a one-cycle gap, and alternating dimensions to catch accumulator leakage between transactions.
Category 3 — Weight Poison & Reset Stress (7)	Combines directed weight patterns engineered to surface silent accumulator corruption with reset assertions timed to each FSM state, to probe recovery correctness.
Category 4 — Reset Behavior (3)	Verifies synchronous reset returns the FSM and register state to a clean baseline from IDLE, immediately after a computation, and from the DONE state.
Category 5 — Illegal Operation / Error Injection (4)	Drives protocol and sequencing violations — illegal register writes, premature or duplicate START assertions, and out-of-range dimensions — to confirm the DUT degrades safely.
Category 6 — Latency & Throughput Performance (5)	Confirms the 2N-cycle latency contract holds at every dimension, and that back-to-back throughput matches the theoretical maximum, with dual signal-level and transaction-level checking.
Register Abstraction Layer (RAL) Built-In Sequences (2)	Runs the UVM RAL framework's built-in reset and access sequences as a register-model health check that gates all subsequent functional testing.

2. Test Case Index

Quick-reference listing of every test case in document order, grouped by category.

ID	Category	Title
TC-001	Cat 1	Full 4×4 Random Matrix Multiply
TC-002	Cat 1	3×3 Subarray

ID	Category	Title
TC-003	Cat 1	2×2 Subarray
TC-004	Cat 1	1×1 Scalar MAC
TC-005	Cat 1	All-Zero Activation Matrix
TC-006	Cat 1	All-Zero Weight Matrix
TC-007	Cat 1	Max int8 Values (Worst-Case Accumulation)
TC-008	Cat 1	Min int8 Values (Negative Accumulation)
TC-009	Cat 1	Mixed Positive and Negative Values
TC-010	Cat 1	Identity Matrix as Weights
TC-011	Cat 2	10+ Consecutive 4×4 Computations, No Gap
TC-012	Cat 2	Back-to-Back With One-Cycle Gap
TC-013	Cat 2	Back-to-Back With Alternating Dimensions
TC-014	Cat 3	Weight Poison: All-Zero Weights
TC-015	Cat 3	Weight Poison: All-127 Weights
TC-016	Cat 3	Weight Poison: Identity Matrix Weights
TC-017	Cat 3	Weight Poison: Checkerboard Pattern
TC-018	Cat 3	Reset Stress: Reset During WEIGHT_LOAD
TC-019	Cat 3	Reset Stress: Reset During PE_CLEAR
TC-020	Cat 3	Reset Stress: Reset During ACTIVATION_FLOW
TC-021	Cat 4	Synchronous Reset From IDLE
TC-022	Cat 4	Reset Then Immediate Restart
TC-034	Cat 4	Reset Asserted While FSM Is in DONE State
TC-023	Cat 5	Write to Read-Only STATUS_REG
TC-024	Cat 5	START Asserted Before DIM_REG Written (dim=0)
TC-025	Cat 5	Double START (Second START Mid-Computation)
TC-026	Cat 5	Invalid Dimension (DIM_REG > 4)
TC-027	Cat 6	Single N=1 Latency Verification
TC-028	Cat 6	Single N=2 Latency Verification
TC-029	Cat 6	Single N=3 Latency Verification
TC-030	Cat 6	Single N=4 Latency Verification (Worst Case)
TC-031	Cat 6	Back-to-Back Throughput at N=4
TC-032	RAL	RAL Built-In: uvm_reg_hw_reset_seq
TC-033	RAL	RAL Built-In: uvm_reg_access_seq

3. Detailed Test Cases

3.1 Category 1 — Basic Functional Correctness

Validates core matrix-multiply correctness across array dimensions, value extremes, and weight/activation patterns, checked against a Python golden reference model.

TC-001 — Full 4×4 Random Matrix Multiply

Stimulus: Constrained-random int8 values for both A (activations) and B (weights). DIM_REG written to 4. CTRL_REG start asserted. Activations driven into left edge of array, staggered by row index (row K delayed K cycles).

Expected Behavior: FSM transitions IDLE → WEIGHT_LOAD → PE_CLEAR → ACTIVATION_FLOW → DONE. STATUS_REG done bit asserts exactly 2N = 8 cycles after activation flow begins. Output buffer holds 16 int32 results.

Scoreboard Check: Python golden model called with same A and B matrices. All 16 output values compared element-by-element. Any mismatch raises uvm_error with full context (expected value, got value, matrix inputs, test name).

Assertions Exercised: no_skip_weight_load — FSM must pass through WEIGHT_LOAD before ACTIVATION_FLOW; pe_clear_one_cycle — pe_clear asserts for exactly one cycle before ACTIVATION_FLOW; result_latency — done asserts exactly 2N cycles after activation flow starts; no_spurious_done — done must not assert while in IDLE

Coverage: cp_dim = full (4), cp_weight_pattern = random, cp_activation_pattern = random

TC-002 — 3×3 Subarray

Stimulus: Directed or constrained-random int8 values. DIM_REG written to 3. Only the top-left 3×3 quadrant of the physical array is active.

Expected Behavior: FSM completes correctly for N=3. Done asserts exactly 2N = 6 cycles after activation flow. Output buffer holds 9 valid int32 results. The 4th row and column of the physical array are idle — their accumulators must not contribute to output.

Scoreboard Check: Golden model called with 3×3 matrices. 9 outputs compared. No values from the inactive 4th row/column should appear.

Assertions Exercised: result_latency — 2N=6 for N=3; pe_clear_one_cycle

Coverage: cp_dim = small (3)

TC-003 — 2×2 Subarray

Stimulus: Directed int8 values for first run (hand-verifiable), then constrained-random. DIM_REG written to 2. This is the smoke-test case — a 2×2 result can be verified by hand before trusting the golden model.

Expected Behavior: Done asserts exactly 2N = 4 cycles after activation flow. 4 valid int32 results in output buffer.

Scoreboard Check: Golden model called with 2×2 matrices. This test is also the target of JasperGold Proof 3 — formal correctness for all 2^32 int8 input combinations. The simulation test exercises the same scenario with specific random seeds.

Assertions Exercised: result_latency — 2N=4 for N=2

Coverage: cp_dim = small (2)

TC-004 — 1×1 Scalar MAC

Stimulus: Directed int8 values (e.g., A=5, B=3, expected output=15). DIM_REG written to 1.

Expected Behavior: Done asserts exactly 2N = 2 cycles after activation flow. Single int32 result in output buffer equal to A×B.

Scoreboard Check: Golden model called with 1×1 matrices. Single value comparison.

Assertions Exercised: result_latency — 2N=2 for N=1 (tightest latency case)

Coverage: cp_dim = scalar (1)

TC-005 — All-Zero Activation Matrix

Stimulus: Directed: all activations = 0, weights constrained-random int8. DIM_REG = 4.

Expected Behavior: All 16 output values must be exactly 0. No partial accumulation from prior runs (pe_clear must have worked).

Scoreboard Check: Golden model returns all-zero matrix. Any nonzero output is a silent data corruption bug — either pe_clear failed or the multiply is wrong.

Assertions Exercised: zero_input_no_accumulate (pe_sva.sv) — accumulator must not change when activation_in == 0

Coverage: cp_activation_pattern = all_zero, cx_dim_x_act = full × all_zero

TC-006 — All-Zero Weight Matrix

Stimulus: Directed: all weights = 0, activations constrained-random int8. DIM_REG = 4.

Expected Behavior: All 16 output values must be exactly 0.

Scoreboard Check: Golden model returns all-zero matrix. Nonzero output means the weight preload or MAC logic is broken.

Assertions Exercised: zero_input_no_accumulate — indirectly, since 0 × anything = 0 accumulated

Coverage: cp_weight_pattern = all_zero, cx_dim_x_weight = full × all_zero

TC-007 — Max int8 Values (Worst-Case Accumulation)

Stimulus: Directed: all activations = 127, all weights = 127. DIM_REG = 4. Worst-case accumulation: each PE accumulates 4 × (127 × 127) = 64,516.

Expected Behavior: All outputs = 64,516 (for 4×4). No overflow — int32 max is ~2.1 billion, so 64,516 fits with enormous headroom. This is the scenario JasperGold Proof 1 formally proves can never overflow.

Scoreboard Check: Golden model returns the expected all-64516 matrix. Any overflow in RTL would produce a wrapped or incorrect int32 value.

Assertions Exercised: Overflow impossibility is formally proven in JasperGold (pe_formal.sv). This simulation test confirms the same property under simulation with the specific worst-case input.

Coverage: cp_weight_pattern = all_max, cp_activation_pattern = all_max, cx_dim_x_weight = full × all_max

TC-008 — Min int8 Values (Negative Accumulation)

Stimulus: Directed: all activations = -128, all weights = -128. DIM_REG = 4. Each PE accumulates 4 × ((-128) × (-128)) = 4 × 16,384 = 65,536. Result is positive (negative × negative = positive).

Expected Behavior: All outputs = 65,536. Signed arithmetic must be handled correctly — the multiply of two negatives must produce a positive int32.

Scoreboard Check: Golden model returns 65,536 for all entries. Sign error in the multiply logic would produce wrong or negative values.

Assertions Exercised: No specific SVA, but scoreboard catches any signed arithmetic mistake immediately.

Coverage: cp_weight_pattern = all_negative, cp_activation_pattern = all_negative

TC-009 — Mixed Positive and Negative Values

Stimulus: Constrained-random with constraint that at least one quadrant of each matrix contains negative values. DIM_REG = 4.

Expected Behavior: Outputs reflect signed cancellation correctly — some entries will be positive, some negative, some zero depending on the random values.

Scoreboard Check: Golden model called with same matrices. This test specifically stresses signed MAC correctness since random all-positive values could accidentally pass a broken sign implementation.

Assertions Exercised: Scoreboard is the primary check here.

Coverage: cp_weight_pattern = random, cp_activation_pattern = random (with signed constraint)

TC-010 — Identity Matrix as Weights

Stimulus: Directed: B = identity matrix (1s on diagonal, 0s elsewhere), A = constrained-random int8. DIM_REG = 4.

Expected Behavior: Output C must equal A exactly — multiplying by the identity matrix is a no-op. This is a self-checking test: if the output doesn't match the input activations, something is wrong with the weight preload or the MAC routing.

Scoreboard Check: Golden model confirms C == A. This test is particularly good at catching PE wiring bugs — if any PE has its weight loaded from the wrong row/column, the identity property breaks.

Assertions Exercised: no_skip_weight_load — weight preload is especially critical for this test to be meaningful

Coverage: cp_weight_pattern = random (identity is a specific pattern, may need its own bin)

3.2 Category 2 — Back-to-Back Transaction Sequencing

Stresses consecutive computations with no gap, a one-cycle gap, and alternating dimensions to catch accumulator leakage between transactions.

TC-011 — 10+ Consecutive 4×4 Computations, No Gap

Stimulus: Constrained-random int8 A and B matrices for each transaction. DIM_REG held at 4 throughout. Immediately after STATUS_REG done asserts, CTRL_REG start is re-asserted with new matrices — no idle cycles between transactions. Run minimum 10 consecutive computations.

Expected Behavior: Each computation produces a correct independent result. The FSM cycles through WEIGHT_LOAD → PE_CLEAR → ACTIVATION_FLOW → DONE for every transaction. pe_clear asserts exactly one cycle before each ACTIVATION_FLOW, zeroing all accumulators before new data flows in.

Scoreboard Check: Golden model called separately for each transaction with that transaction's specific A and B. All 10+ results checked independently. Any result that contains values from a prior transaction's accumulation is caught immediately — this is the primary accumulator leakage detection test.

Assertions Exercised: pe_clear_one_cycle — must fire on every transaction, not just the first; result_latency — 2N=8 must hold for every transaction in the sequence, not just the first

Coverage: cp_back_to_back = no_gap (0)

TC-012 — Back-to-Back With One-Cycle Gap

Stimulus: Same as TC-011 but one idle cycle inserted between DONE asserting and the next CTRL_REG start. 10+ consecutive transactions.

Expected Behavior: Identical correct results to TC-011. The one-cycle gap should make no functional difference — this test verifies the DUT is not accidentally dependent on a minimum recovery time that isn't in the spec.

Scoreboard Check: Same as TC-011 — golden model per transaction, independent comparison.

Assertions Exercised: pe_clear_one_cycle; no_spurious_done — done must not re-assert during the idle gap cycle

Coverage: cp_back_to_back = one_cycle (1)

TC-013 — Back-to-Back With Alternating Dimensions

Stimulus: Directed sequence: 4×4 computation, immediately followed by 2×2, immediately followed by 4×4 again. DIM_REG rewritten before each transaction. Constrained-random int8 values for each.

Expected Behavior: Each computation produces the correct result for its own dimension. The 2×2 result must not be contaminated by the preceding 4×4 computation's accumulator values — specifically, PEs [0][0], [0][1], [1][0], [1][1] must be cleanly zeroed by pe_clear even though they accumulated 4 cycles worth of data in the prior 4×4 run. The 4×4 run following the 2×2 must also be clean.

Scoreboard Check: Golden model called with the correct dimension for each transaction. The 2×2 golden model call uses only the top-left 2×2 submatrix of whatever values were driven.

Assertions Exercised: pe_clear_one_cycle — especially critical here since the same PEs are reused across different dimension transactions; result_latency — must match 2N for each respective N (8, 4, 8 cycles)

Coverage: cp_back_to_back = no_gap, cp_dim = full and small crossed together — hits cx_dim_x_weight and cx_dim_x_act bins for multiple dimensions in sequence

3.3 Category 3 — Weight Poison & Reset Stress

Combines directed weight patterns engineered to surface silent accumulator corruption with reset assertions timed to each FSM state, to probe recovery correctness.

TC-014 — Weight Poison: All-Zero Weights

Stimulus: Directed: all weights = 0, activations constrained-random int8. DIM_REG = 4. Run this immediately after a prior computation that had large nonzero weight values — making this the hardest possible test for accumulator leakage via the weight path.

Expected Behavior: All 16 outputs must be exactly 0. Zero weights multiplied by any activation = 0, accumulated N times = 0. Any nonzero output means either pe_clear failed, weight preload is broken, or the MAC logic is not correctly zeroing when weight = 0.

Scoreboard Check: Golden model returns all-zero matrix. A nonzero output here is unambiguous — there is no “close but slightly wrong” for an all-zero weight matrix. This makes it the highest-signal silent corruption detector in the entire test plan.

Assertions Exercised: zero_input_no_accumulate (pe_sva.sv) — weight = 0 means activation × weight = 0, accumulator must not change

Coverage: cp_weight_pattern = all_zero, cx_dim_x_weight = full × all_zero

TC-015 — Weight Poison: All-127 Weights

Stimulus: Directed: all weights = 127, activations constrained-random int8. DIM_REG = 4. Worst-case accumulation path — every PE is accumulating at maximum weight magnitude every cycle.

Expected Behavior: Each output = sum of (activation[row][k] × 127) for k = 0 to N-1. The golden model computes this exactly. This test stresses the accumulator's ability to handle large repeated additions without overflow or saturation.

Scoreboard Check: Golden model called with all-127 weight matrix. Any rounding, saturation, or overflow artifact in the RTL accumulator produces a mismatch. This pairs with TC-007 from Category 1 but with random activations rather than max activations — different stress profile.

Assertions Exercised: Scoreboard is primary. No specific SVA targets this directly — overflow impossibility is formally proven in JasperGold Proof 1.

Coverage: cp_weight_pattern = all_max, cx_dim_x_weight = full × all_max

TC-016 — Weight Poison: Identity Matrix Weights

Stimulus: Directed: B = identity matrix, A = constrained-random int8. DIM_REG = 4. Run immediately after a prior computation with large nonzero weights — stressing weight preload correctness since the identity matrix requires exact 0s and 1s in the right positions.

Expected Behavior: Output C must equal A exactly. This tests weight preload routing — if any PE receives its weight from the wrong row or column during preload, the identity property breaks and the output will differ from the activations. This is a very sensitive PE wiring test.

Scoreboard Check: Golden model confirms C == A. Any PE wiring bug, any weight loaded into the wrong PE, or any preload sequencing error in the controller will produce a mismatch here that would be much harder to detect with random weights.

Assertions Exercised: no_skip_weight_load — weight preload must complete fully before activation flow; partial preload destroys the identity property

Coverage: cp_weight_pattern = random (may want a dedicated identity bin added to the covergroup)

TC-017 — Weight Poison: Checkerboard Pattern

Stimulus: Directed: weights alternate +127 and -127 in a checkerboard pattern (B[i][j] = 127 if (i+j) is even, -127 if (i+j) is odd). Activations constrained-random int8. DIM_REG = 4.

Expected Behavior: Each output reflects alternating sign accumulation — partial products are large positive and large negative values summed together. This stresses the signed arithmetic path specifically, since the cancellation of large positive and negative values is where two's complement bugs most commonly appear.

Scoreboard Check: Golden model called with checkerboard weight matrix. A sign error in the MAC logic that happens to cancel out under all-positive or all-negative inputs will show up clearly here because the expected output requires correct handling of both signs simultaneously.

Assertions Exercised: Scoreboard is primary. No specific SVA — the signed arithmetic correctness is a datapath property best caught by the reference model comparison.

Coverage: cp_weight_pattern = all_negative (partial — checkerboard has both signs; may want dedicated bin)

TC-018 — Reset Stress: Reset During WEIGHT_LOAD

Stimulus: Begin a 4×4 computation normally. Assert synchronous reset exactly when the FSM is in WEIGHT_LOAD state (mid-preload, some weights loaded, some not). Release reset. Immediately start a new clean 4×4 computation with known values.

Expected Behavior: FSM returns to IDLE on reset. All partially loaded weights are irrelevant — pe_clear on the next computation must zero all accumulators. The new computation must produce a result based solely on its own weight and activation values, with no contamination from the partially-loaded weights.

Scoreboard Check: No output expected from the aborted computation. The subsequent clean computation is checked against the golden model. Any output value that reflects the partially-loaded weights from before the reset is a reset recovery bug.

Assertions Exercised: no_spurious_done — done must not assert after reset during WEIGHT_LOAD; pe_clear_one_cycle — must fire correctly on the post-reset computation

Coverage: cp_back_to_back — reset-then-restart is a variant of back-to-back with a forced interruption

TC-019 — Reset Stress: Reset During PE_CLEAR

Stimulus: Begin a 4×4 computation. Assert synchronous reset exactly when FSM is in PE_CLEAR state — the single cycle where pe_clear is asserted. This is the tightest timing

window to hit and specifically tests whether a reset during the clear cycle leaves accumulators in a partially-zeroed state.

Expected Behavior: FSM returns to IDLE. Accumulators may be partially zeroed or fully zeroed depending on reset timing — it doesn't matter, because the next computation's pe_clear must zero them completely regardless. The subsequent computation must be clean.

Scoreboard Check: Same as TC-018 — no output from aborted computation, next computation verified against golden model.

Assertions Exercised: pe_clear_one_cycle — after reset, the next computation's pe_clear must still assert for exactly one cycle; no_spurious_done

Coverage: Reset during PE_CLEAR is a dedicated coverage point — consider adding an explicit cp_reset_state coverpoint with bins for each FSM state

TC-020 — Reset Stress: Reset During ACTIVATION_FLOW

Stimulus: Begin a 4×4 computation. Assert synchronous reset midway through ACTIVATION_FLOW — some activations have already flowed through, partial results are accumulating inside PEs. This is the highest-yield bug surface according to the master plan because mid-computation accumulator values are at their largest and most likely to leak.

Expected Behavior: FSM returns to IDLE. Partial accumulator values from the in-progress computation are discarded. The subsequent clean computation starts fresh, pe_clear fires, and the result is correct.

Scoreboard Check: No output from the aborted computation. Subsequent computation verified against golden model. The master plan (Page 13, Rule 17) states this sequence is expected to find at least one real bug — accumulator leakage after a mid-computation reset is a historically common RTL defect.

Assertions Exercised: no_spurious_done — done must not fire after reset mid-ACTIVATION_FLOW; pe_clear_one_cycle — post-reset computation must trigger pe_clear correctly

Coverage: cp_reset_state = ACTIVATION_FLOW bin

3.4 Category 4 — Reset Behavior

Verifies synchronous reset returns the FSM and register state to a clean baseline from IDLE, immediately after a computation, and from the DONE state.

TC-021 — Synchronous Reset From IDLE

Stimulus: DUT sitting in IDLE state (no computation in progress). Assert synchronous reset for one cycle. Release reset. Verify DUT returns cleanly to IDLE with all registers and accumulators at reset values.

Expected Behavior: FSM remains in IDLE after reset releases. STATUS_REG done bit deasserts if it was previously set. DIM_REG and CTRL_REG return to reset values (0). All PE accumulators zeroed. DUT ready to accept a new legal transaction immediately.

Scoreboard Check: RAL built-in sequence uvm_reg_hw_reset_seq is run after reset releases — walks every register and verifies reset values match the spec (DIM_REG = 0, CTRL_REG = 0, STATUS_REG = 0). This is a free check that comes with the RAL model.

Assertions Exercised: no_spurious_done — done must not assert during or after reset from IDLE; FSM must remain in IDLE — no_skip_weight_load confirms no illegal state transition occurs

Coverage: cp_reset_state = IDLE bin (dedicated coverpoint recommended)

TC-022 — Reset Then Immediate Restart

Stimulus: Complete a full 4×4 computation successfully. Assert reset for one cycle. Release reset. On the very next cycle after reset releases, write DIM_REG and assert CTRL_REG start — minimum possible recovery time. Run the new computation with known directed values.

Expected Behavior: DUT accepts the new transaction immediately after reset without requiring any additional idle cycles. FSM transitions correctly through WEIGHT_LOAD → PE_CLEAR → ACTIVATION_FLOW → DONE. Result is correct for the new inputs, completely independent of the prior computation.

Scoreboard Check: Golden model called with the post-reset computation's matrices. Any contamination from the pre-reset computation produces a mismatch. This test specifically catches hidden minimum-recovery-time assumptions in the controller FSM that aren't in the spec.

Assertions Exercised: pe_clear_one_cycle — must fire correctly even on the immediate post-reset computation; result_latency — 2N=8 must hold on the first post-reset computation; no_spurious_done — done from prior computation must not re-assert after reset

Coverage: cp_back_to_back = no_gap combined with reset — consider a dedicated cp_reset_recovery coverpoint

TC-034 — Reset Asserted While FSM Is in DONE State

Stimulus: Run a complete legal 4×4 computation to completion. Allow STATUS_REG done bit to assert and hold — do not immediately re-assert START. Assert synchronous reset for one cycle while the FSM is sitting in DONE state with done asserted. Release reset.

Expected Behavior: FSM transitions from DONE back to IDLE on reset. STATUS_REG done bit deasserts — it must not remain asserted after reset since it reflects the FSM's DONE state which no longer exists. All PE accumulators zeroed. DUT ready to accept a new legal transaction. A subsequent clean computation is run immediately after reset release and must produce a correct result.

Scoreboard Check: Two checks. First, negative check: done must deassert within one cycle of reset asserting — any persistence of the done bit after reset is a reset recovery bug specific to the DONE state. Second, positive check: the post-reset computation is verified against the golden model, confirming the DUT's state was fully cleaned up by reset and the new computation runs independently.

Assertions Exercised: no_spurious_done — done must not persist after reset; must not re-assert during the post-reset idle period; pe_clear_one_cycle — post-reset computation must trigger pe_clear correctly even though the prior computation completed normally; result_latency — 2N=8 must hold on the post-reset computation

Coverage: cp_reset_state = done_state — this is the specific bin that required TC-034 to be added. Without this test, the cp_reset_state covergroup would have a permanently empty bin for the DONE state, and Phase 3 coverage closure would fail.

3.5 Category 5 — Illegal Operation / Error Injection

Drives protocol and sequencing violations — illegal register writes, premature or duplicate START assertions, and out-of-range dimensions — to confirm the DUT degrades safely.

TC-023 — Write to Read-Only STATUS_REG

Stimulus: Using the RAL model, attempt to write a value to STATUS_REG (which is declared “RO” in mmu_reg_model.sv). The RAL model flags this as a protocol violation automatically. The AXI driver still physically drives the write transaction onto the bus to test the hardware response.

Expected Behavior: DUT ignores the write completely. STATUS_REG value is unchanged — if done was 0 before the write, it remains 0; if done was 1, it remains 1. No spurious state change, no FSM transition, no output produced.

Scoreboard Check: Negative check: scoreboard verifies no output appears during the illegal operation window. After the illegal write, a legal computation is run and verified against the golden model — confirming the DUT's internal state was not corrupted by the illegal write.

Assertions Exercised: axi_awvalid_stable — AXI handshake must still complete correctly even for the illegal write; no_spurious_done — done must not assert as a result of the illegal write

Coverage: Dedicated error injection coverpoint — cp_error_type = status_reg_write

TC-024 — START Asserted Before DIM_REG Written (dim=0)

Stimulus: Directed: skip writing DIM_REG entirely (reset value = 0). Assert CTRL_REG start immediately. DIM_REG = 0 is an illegal dimension — no valid N×N computation exists for N=0.

Expected Behavior: FSM stays in IDLE or handles the zero dimension gracefully — no hang, no infinite loop, no output produced. The exact behavior (stay in IDLE vs brief excursion and return) must be defined in the spec doc and the assertion must match that definition.

Scoreboard Check: Negative check: no output produced. No done assertion. After the illegal start attempt, DUT is driven with a valid DIM_REG value and a new START — the subsequent legal computation must succeed and produce a correct result.

Assertions Exercised: no_spurious_done — done must never assert when dim=0; FSM no-deadlock property — FSM must eventually return to IDLE given no valid start sequence

Coverage: cp_error_type = premature_start

TC-025 — Double START (Second START Mid-Computation)

Stimulus: Begin a legal 4×4 computation. While the FSM is in ACTIVATION_FLOW state, assert CTRL_REG start a second time. The first computation is still in progress.

Expected Behavior: DUT ignores the second START entirely. The first computation runs to completion, done asserts exactly 2N=8 cycles after its own activation flow began, and the output buffer contains the correct result for the first computation only. No duplication, no corruption, no premature termination of the first computation.

Scoreboard Check: Negative check during the illegal window — no spurious second done assertion while first computation is running. Positive check after first computation completes — output matches golden model for the first computation's matrices. The second START must have had zero effect.

Assertions Exercised: no_spurious_done — only one done assertion per computation; result_latency — first computation's latency must be unaffected by the second START

Coverage: cp_error_type = double_start

TC-026 — Invalid Dimension (DIM_REG > 4)

Stimulus: Directed: write DIM_REG = 5 (or 6, 7 — values that exceed the physical 4×4 array). Assert CTRL_REG start. The 3-bit DIM_REG field can hold values 0–7, but only 1–4 are valid per the spec.

Expected Behavior: DUT clamps to maximum valid dimension (4) or ignores the start entirely — the exact behavior must be defined in the spec doc before this test is written. No crash, no hang, no accessing PE indices that don't exist (which would be an out-of-bounds generate block access in RTL). If the DUT clamps to N=4, the output must be correct for a 4×4 computation.

Scoreboard Check: If clamping behavior is spec'd: scoreboard checks output against 4×4 golden model. If ignore behavior is spec'd: negative check, no output produced. Either way, subsequent legal transaction must succeed.

Assertions Exercised: no_spurious_done — if DUT ignores the illegal dimension, done must not assert; FSM must not deadlock — must return to IDLE regardless of the illegal dimension value

Coverage: cp_error_type = invalid_dim

3.6 Category 6 — Latency & Throughput Performance

Confirms the 2N-cycle latency contract holds at every dimension, and that back-to-back throughput matches the theoretical maximum, with dual signal-level and transaction-level checking.

TC-027 — Single N=1 Latency Verification

Stimulus: Directed: DIM_REG = 1, known int8 scalar values for A and B. Single computation. Cycle counter started the moment activation flow begins.

Expected Behavior: Done asserts exactly 2N = 2 cycles after activation flow starts. This is the tightest latency case — only 2 cycles. An off-by-one in the controller's DONE timing logic is most visible here because the window is so narrow.

Scoreboard Check: Samarth's latency_checker inside mmu_scoreboard.sv records the cycle CTRL_REG start fires and the cycle STATUS_REG done asserts, then asserts observed_latency == 2. Jad's mmu_perf_checker.sv fires simultaneously at the signal level. Both must agree.

Assertions Exercised: result_latency — 2N=2 for N=1; mmu_perf_checker.sv signal-level assertion fires if done is even one cycle early or late

Coverage: cp_dim = scalar (1) combined with performance coverpoint

TC-028 — Single N=2 Latency Verification

Stimulus: Directed: DIM_REG = 2, known int8 values. Single computation.

Expected Behavior: Done asserts exactly 2N = 4 cycles after activation flow starts. This is the dimension used in JasperGold Proof 3 — the simulation test and the formal proof cover the same scenario from complementary angles.

Scoreboard Check: latency_checker asserts observed_latency == 4. mmu_perf_checker.sv asserts the same at signal level.

Assertions Exercised: result_latency — 2N=4 for N=2

Coverage: cp_dim = small (2) with performance coverpoint

TC-029 — Single N=3 Latency Verification

Stimulus: Directed: DIM_REG = 3, known int8 values. Single computation.

Expected Behavior: Done asserts exactly 2N = 6 cycles after activation flow starts.

Scoreboard Check: latency_checker asserts observed_latency == 6. mmu_perf_checker.sv confirms at signal level.

Assertions Exercised: result_latency — 2N=6 for N=3

Coverage: cp_dim = small (3) with performance coverpoint

TC-030 — Single N=4 Latency Verification (Worst Case)

Stimulus: Directed: DIM_REG = 4, known int8 values. Single computation. This is the most important latency test — the worst-case timing the controller must meet.

Expected Behavior: Done asserts exactly 2N = 8 cycles after activation flow starts. Any pipeline register miscounting or FSM off-by-one in the ACTIVATION_FLOW → DONE transition shows up here as a one-cycle deviation.

Scoreboard Check: latency_checker asserts observed_latency == 8. mmu_perf_checker.sv asserts at signal level. These two checkers operate at different abstraction levels intentionally — both must pass.

Assertions Exercised: result_latency — 2N=8 for N=4; pe_clear_one_cycle — pe_clear's single cycle must be correctly accounted for in the 8-cycle total

Coverage: cp_dim = full (4) with performance coverpoint — highest priority bin

TC-031 — Back-to-Back Throughput at N=4

Stimulus: Run 10+ consecutive N=4 computations with zero gap between them — immediately re-asserting START after each DONE. Constrained-random int8 matrices for each transaction. Cycle counter running continuously across all transactions.

Expected Behavior: Each individual computation completes in exactly 2N=8 cycles. Back-to-back throughput matches the theoretical maximum — the DUT should not require any additional stall cycles between consecutive transactions beyond what the FSM naturally requires (WEIGHT_LOAD cycles). Any hidden stall or pipeline bubble between transactions reduces throughput below the theoretical maximum and is caught here.

Scoreboard Check: Two simultaneous checks. Functional: golden model called per transaction, all 10+ outputs verified independently (same as TC-011). Performance: perf_sequences.sv records total cycle count across all 10+ transactions and computes actual throughput. perf_report.md documents actual vs expected. Any throughput deviation from theoretical maximum is logged in BUGS.md with root cause.

Assertions Exercised: mmu_perf_checker.sv — throughput assertions fire if back-to-back gap exceeds expected; pe_clear_one_cycle — must fire cleanly on every transaction in the sequence; result_latency — 2N=8 on every individual transaction

Coverage: cp_back_to_back = no_gap, cp_dim = full (4), throughput coverage bin in perf_sequences.sv

3.7 Register Abstraction Layer (RAL) Built-In Sequences

Runs the UVM RAL framework's built-in reset and access sequences as a register-model health check that gates all subsequent functional testing.

TC-032 — RAL Built-In: uvm_reg_hw_reset_seq

Stimulus: No custom stimulus required. The UVM RAL framework's built-in uvm_reg_hw_reset_seq is invoked directly on the register model after a synchronous reset cycle. It automatically walks every register in the model — DIM_REG, CTRL_REG, STATUS_REG — and reads each one back via the AXI-Lite interface.

Expected Behavior: Every register returns its defined reset value: DIM_REG → 0x0 (3-bit field, reset = 0), CTRL_REG → 0x0 (1-bit field, reset = 0), STATUS_REG → 0x0 (1-bit done field, reset = 0). No computation is triggered. The DUT remains in IDLE throughout.

Scoreboard Check: The RAL model performs the comparison internally — it knows the expected reset value of every field from the register definitions in mmu_reg_model.sv. Any mismatch between the read-back value and the defined reset value raises a uvm_error automatically. This is a register model correctness test.

Assertions Exercised: no_spurious_done — done must not assert at any point during this sequence; axi_awvalid_stable — AXI read handshake must complete correctly for every register read

Coverage: cp_reset_state = IDLE. Also exercises the AXI read path for all three registers — contributes to cp_error_type indirectly since STATUS_REG is read-only and this sequence confirms its reset value without writing to it.

Note: This is the first test run in Phase 3 integration, before any other sequences. If this fails, the register model or AXI-Lite slave has a fundamental bug that would corrupt every subsequent test. Fix this before moving on.

TC-033 — RAL Built-In: uvm_reg_access_seq

Stimulus: No custom stimulus required. The UVM RAL framework's built-in uvm_reg_access_seq is invoked on the register model. It automatically walks every RW register — DIM_REG and CTRL_REG — writes a pattern value, reads it back, and verifies the round-trip. STATUS_REG is declared RO in the RAL model and is skipped for write operations automatically.

Expected Behavior: DIM_REG and CTRL_REG return the written value when read back. STATUS_REG is not written — the RAL model enforces this via the “RO” access type definition. The DUT remains behaviorally stable — writing to DIM_REG with CTRL_REG unasserted does not trigger a computation.

Scoreboard Check: The RAL model performs write-then-read comparison internally for every RW field. Any field that does not retain its written value raises a uvm_error. This catches a class of RTL bugs — registers that appear to accept writes but silently reset to zero, or registers whose bits are wired incorrectly such that only some bits survive a round-trip.

Assertions Exercised: axi_awvalid_stable — AXI write handshake must complete for every RW register write; axi_wvalid_with_awvalid — write data valid must accompany write address

valid for each transaction; no_spurious_done — a CTRL_REG write during this sequence must not accidentally trigger a computation and assert done

Coverage: Contributes to cp_error_type = status_reg_write indirectly — the RAL model's automatic skip of the STATUS_REG write confirms the read-only enforcement is working at the model layer. Full write-path AXI coverage for DIM_REG and CTRL_REG.

Note: This is the second test run in Phase 3, immediately after TC-032. Together TC-032 and TC-033 form the register model health check that gates all subsequent testing. Both must pass cleanly before any functional sequences are run.

4. Negative Testing — Assertion Validation via Force Injection

The TC-035 series intentionally forces internal DUT signals into illegal states using SystemVerilog force/release to confirm each assertion actually fires when its property is violated. Without this step, an assertion that has never fired could simply be miswritten or unbound rather than genuinely passing. Each variant targets one or two specific assertions from the SVA reference in Section 5, and each is followed by a clean, legal computation to confirm the DUT recovers correctly after the force is released.

TC-035a

Forces: FSM state forced IDLE→PE_CLEAR, skipping WEIGHT_LOAD

Validates: B1

Detail: With FSM in IDLE and START about to assert, force state = PE_CLEAR for one cycle on the same edge START would normally trigger WEIGHT_LOAD. Release the force on the next cycle. The forced state is illegal per the FSM's designed transition map. B1 (no_skip_weight_load) must fire exactly once, on the cycle following the forced state change. After release, a clean legal computation is run and must pass the standard positive check.

TC-035b

Forces: pe_clear held for 2 cycles

Validates: B2

Detail: During a legal computation's PE_CLEAR state, force pe_clear = 1 to persist for one additional cycle beyond its normal one-cycle assertion. B2 (pe_clear_one_cycle) must fire exactly once, on the second cycle of the forced hold. Follow-on clean computation verified against golden model.

TC-035c

Forces: done forced at cycle 2N-1 (one cycle early)

Validates: B3, D1

Detail: During a legal N=4 computation, force done = 1 at cycle 2N-1 (7 cycles after activation_flow_start instead of 8). Release immediately after. Both B3 (controller-level) and D1 (top-level signal) expect done at exactly cycle 2N — an early assertion violates both simultaneously. If only one fires, that's flagged as its own anomaly worth investigating.

TC-035d

Forces: done forced late (suppressed until cycle 2N+1)

Validates: B3, D1

Detail: During a legal N=4 computation, suppress the natural done assertion at cycle 2N (force it to 0), then release the force at cycle 2N+1, allowing the RTL's natural logic to assert it one cycle late. Same properties as TC-035c, opposite direction — confirms the properties catch violations in both directions.

TC-035e

Forces: done forced while state==IDLE

Validates: B4

Detail: With the FSM sitting in IDLE (no computation in progress), force done = 1 for one cycle. Release. B4 (no_spurious_done) must fire exactly once, on the forced cycle. This is also a good regression sanity check — if B4 doesn't fire here, the assertion binding itself may be broken.

TC-035f

Forces: PE accumulator forced to increment on zero activation (1 PE instance)

Validates: C1

Detail: During a computation where a specific PE instance (e.g., systolic_array.pe_inst[1][2]) receives activation_in == 0 on a given cycle, force that specific PE's acc register to increment by 1 on that same cycle. C1 must fire on PE[1][2] specifically, and only PE[1][2] — the other 15 instances must show zero violations for this cycle, validating that bind created independent assertion instances per PE.

TC-035g

Forces: Extra WEIGHT_LOAD stall cycle in back-to-back sequence

Validates: D2

Detail: During a back-to-back sequence (modeled on TC-011), after done asserts and a new START is pending, force the FSM to remain in WEIGHT_LOAD for one cycle longer than its normal duration before transitioning to PE_CLEAR. D2 (back_to_back_throughput) must fire exactly once, on the cycle where activation_flow_start is observed late relative to the expected window.

5. SVA Assertion Reference

9 assertions are bound across the design: AXI-Lite protocol checks (A-prefix), controller FSM and timing properties (B-prefix), per-PE datapath checks bound to every processing-element instance (C-prefix), and signal-level performance checks bound at the top level (D-prefix). "Exercised By" lists the positive-mode test cases expected to pass through each property without firing; "Targeted By" lists the mutation tests in Section 4 designed to deliberately trigger it.

A1 — axi_awvalid_stable

File: axi_lite_sva.sv

Property:

```
@(posedge clk) awvalid && !awready |=> awvalid;
```

Checking: Once the AXI write address valid signal asserts, it must hold until the slave acknowledges it with awready. A master that drops awvalid before the handshake completes is violating the AXI-Lite protocol — the slave may have missed the transaction entirely.

Exercised By (Positive Mode): TC-001 through TC-010 (every functional test writes DIM_REG and CTRL_REG via AXI-Lite); TC-032, TC-033 (RAL sequences — highest density of awvalid/awready handshakes)

Targeted By (Negative Mode): TC-023 + TC-035: deliberately drop awvalid one cycle before awready would assert during the STATUS_REG write attempt.

Frequency: Once per AXI write transaction

A2 — axi_wvalid_with_awvalid

File: axi_lite_sva.sv

Property:

```
@(posedge clk) awvalid |-> wvalid;
```

Checking: In AXI-Lite, write address and write data are sent simultaneously. If awvalid asserts without wvalid, the slave has a write address but no data — undefined behavior. This property enforces the simultaneous assertion requirement.

Exercised By (Positive Mode): TC-001 through TC-010 (every register write drives awvalid and wvalid simultaneously); TC-032, TC-033 (high-density register write traffic)

Targeted By (Negative Mode): TC-025 + TC-035: drive awvalid without wvalid on the second START write attempt to test handling of a malformed write.

Frequency: Once per AXI write transaction

B1 — no_skip_weight_load

File: mmu_controller_sva.sv

Property:

```
@(posedge clk) (state == IDLE) && start |=> (state == WEIGHT_LOAD);
```

Checking: The FSM must always enter WEIGHT_LOAD as the first state after START — it cannot jump directly to PE_CLEAR or ACTIVATION_FLOW. A controller bug that skips weight

loading would produce outputs based on garbage PE register values — a silent catastrophic failure.

Exercised By (Positive Mode): TC-001 through TC-013 (every functional test asserts START from IDLE); TC-022 (reset then immediate restart); TC-034 (post-DONE reset then restart)

Targeted By (Negative Mode): TC-035a: force state = PE_CLEAR while in IDLE with start asserted, bypassing WEIGHT_LOAD.

Frequency: Once per START assertion

B2 — pe_clear_one_cycle

File: mmu_controller_sva.sv

Property:

```
@(posedge clk) $rose(pe_clear) |=> !pe_clear;
```

Checking: pe_clear must assert for exactly one cycle. If it holds for two or more cycles, the controller's PE_CLEAR state is longer than spec, violating the 2N latency contract. A pe_clear timing bug causes both latency violations and potential accumulator corruption.

Exercised By (Positive Mode): TC-001 through TC-013 (every computation triggers exactly one pe_clear pulse); TC-011, TC-012, TC-013 (back-to-back sequences); TC-018, TC-019, TC-020 (reset stress — pe_clear must still be exactly one cycle post-reset)

Targeted By (Negative Mode): TC-035b: force pe_clear to remain asserted for two consecutive cycles.

Frequency: Once per computation

B3 — result_latency

File: mmu_controller_sva.sv

Property:

```
@(posedge clk) $rose(activation_flow_start) |-> ##[2*N:2*N] $rose(done);
```

Checking: Done must assert exactly 2N cycles after activation flow begins — not 2N-1, not 2N+1. This is the pipelined latency contract locked in the master plan, checked here at the signal level and simultaneously at the transaction level by Samarth's latency_checker.

Exercised By (Positive Mode): TC-027, TC-028, TC-029, TC-030 (N=1 through N=4 latency verification); TC-011, TC-031 (back-to-back — must hold on every computation in the chain)

Targeted By (Negative Mode): TC-035c (done one cycle early) and TC-035d (done one cycle late).

Frequency: Once per computation

B4 — no_spurious_done

File: mmu_controller_sva.sv

Property:

```
@(posedge clk) (state == IDLE) |-> !done;
```

Checking: Done must never assert while the FSM is in IDLE. A spurious done assertion from IDLE could cause the testbench to read garbage from the output buffer, or trigger a false pass in a poorly written scoreboard.

Exercised By (Positive Mode): TC-021, TC-022 (reset tests — FSM in IDLE, done must stay low); TC-032, TC-033 (RAL sequences — FSM stays in IDLE throughout); TC-024 (premature START with dim=0); all Category 1 tests (between transactions FSM returns to IDLE)

Targeted By (Negative Mode): TC-035e: force done = 1 while state == IDLE.

Frequency: Every IDLE cycle (highest trigger-count assertion in the project)

C1 — zero_input_no_accumulate

File: pe_sva.sv (×N×N instances)

Property:

```
@(posedge clk) (activation_in == 0) |-> (acc == $past(acc));
```

Checking: When the activation flowing into a PE is zero, the accumulator must not change — because 0 × weight = 0 regardless of weight value. If the accumulator changes when activation_in is zero, either the MAC logic is broken or there is a spurious clock enable issue. Bound to every PE instance — 16 simultaneous copies in a 4×4 array.

Exercised By (Positive Mode): TC-005 (all-zero activation matrix — every PE receives zero every cycle); TC-010 (identity weights with random activations — off-diagonal PEs receive zero for specific cycles); TC-014 (weight poison all-zero — zero weight × any activation = 0)

Targeted By (Negative Mode): TC-035f: force a specific PE's accumulator (e.g. PE[1][2]) to increment by 1 during a zero-activation cycle.

Frequency: Every cycle any PE receives a zero activation, × number of PE instances

D1 — signal_level_latency

File: mmu_perf_checker.sv

Property:

```
@(posedge clk) $rose(activation_flow_start) |-> ##[2*N:2*N] $rose(done);
```

Checking: Jad's signal-level counterpart to B3. B3 is bound to mmu_controller.sv and checks internal controller signals; D1 is bound to mmu_top.sv and checks the top-level interface signals the testbench actually observes. A pipeline register bug between controller-internal done and top-level done would pass B3 but fail D1 — the deliberate redundancy catches this class of bug.

Exercised By (Positive Mode): TC-027 through TC-030 (all four latency verification tests); TC-031 (back-to-back throughput — D1 must hold on every computation at the top-level signal)

Targeted By (Negative Mode): TC-035c and TC-035d, applied at the top-level done signal. If only B3 or only D1 fires, that signals a controller-to-top-level path discrepancy worth logging.

Frequency: Once per computation

D2 — back_to_back_throughput

File: mmu_perf_checker.sv

Property:

```
@(posedge clk) $rose(done) && start_pending |->
##[weight_load_cycles:weight_load_cycles] $rose(activation_flow_start);
```

Checking: In a back-to-back sequence, after done asserts and a new START is issued, the next activation flow must begin within exactly weight_load_cycles — the time needed to preload

new weights. Any additional stall cycles beyond WEIGHT_LOAD are a throughput bug — the signal-level enforcement of the theoretical maximum throughput claim.

Exercised By (Positive Mode): TC-011 (10+ back-to-back N=4 computations, no gap); TC-031 (dedicated throughput test); TC-013 (alternating dimensions — D2 must hold even when WEIGHT_LOAD duration changes)

Targeted By (Negative Mode): TC-035g: force the FSM to remain in WEIGHT_LOAD for one extra cycle beyond spec in a back-to-back sequence.

Frequency: Once per back-to-back transaction boundary

6. Functional Coverage Plan

8 covergroups (including two cross-coverage groups) measure verification closure: matrix dimension, weight and activation pattern character, dimension crossed with each pattern type, inter-transaction gap, error-injection scenario coverage, and the FSM state present at the moment of reset. All bins target 90% or greater closure before Phase 3 sign-off.

cp_dim — Matrix Dimension

Purpose: Ensures the DUT is exercised at every meaningful array size, not just the default full 4×4. A bug in the controller's loop bounds for N=2 would never appear in a test suite that only runs N=4.

Bins:

- scalar = {1} — 1×1 scalar MAC edge case
- small = {2, 3} — 2×2 and 3×3 partial array operation
- full = {4} — 4×4 full physical array

Target: All 3 bins ≥ 90%. scalar and full hit by directed tests (TC-004, TC-001). small requires constrained-random sequences with DIM_REG constrained to {2,3}.

cp_weight_pattern — Weight Matrix Character

Purpose: Ensures the weight preload path and MAC logic are exercised with qualitatively different weight profiles — not just random values that all look similar. A silent bug that only manifests with zero or negative weights would be invisible to purely random generation.

Bins:

- all_zero = {ZERO}
- all_max = {MAX} (127)
- all_negative = {NEG} (-128)
- identity = {IDENTITY}
- checkerboard = {CHECK}
- random = {RAND}

Target: All 6 bins ≥ 90%. Directed bins hit by TC-005, TC-007, TC-008, TC-016, TC-017 and weight_poison_seq. random hit by TC-001 through TC-004.

cp_activation_pattern — Activation Matrix Character

Purpose: Same principle as cp_weight_pattern but on the activation side. The activation path is structurally different — weights are preloaded and stationary, activations flow dynamically cycle by cycle. Bugs in stagger/skew logic that only appear under specific activation patterns would be missed by random-only generation.

Bins:

- all_zero = {ZERO}
- all_max = {MAX} (127)
- all_negative = {NEG} (-128)
- random = {RAND}

Target: All 4 bins ≥ 90%. Hit by TC-005, TC-007, TC-008, and constrained-random Category 1 tests.

cx_dim_x_weight — Cross: Dimension × Weight Pattern

Purpose: Proves every weight pattern was exercised at every array dimension — not just at N=4. A controller bug mishandling weight preload for N=2 with all-zero weights would only show up in the small × all_zero cell of this cross. Without it, 100% on both individual coverpoints could still miss that combination.

Bins:

- 3 dim bins × 6 weight bins = 18 total bins

Target: All 18 bins ≥ 90%. Constrained-random naturally hits most cells; scalar × identity and scalar × checkerboard are hardest to hit by chance — may need directed sequences in Phase 3.

cx_dim_x_act — Cross: Dimension × Activation Pattern

Purpose: Same rationale as cx_dim_x_weight but on the activation path — proves the activation flow logic (stagger timing, data driver, PE input wiring) was exercised with qualitatively different activation profiles at every dimension.

Bins:

- 3 dim bins × 4 activation bins = 12 total bins

Target: All 12 bins ≥ 90%.

cp_back_to_back — Inter-Transaction Gap

Purpose: Ensures state retention correctness (pe_clear behavior) is verified across different gap sizes between consecutive transactions. A zero-gap sequence stresses the DUT maximally; a multi-cycle gap gives it breathing room. Both must work and both must be proven exercised.

Bins:

- no_gap = {0}
- one_cycle = {1}
- multi = {[2:10]}

Target: All 3 bins ≥ 90%. Hit by TC-011 (no_gap), TC-012 (one_cycle), and constrained-random sequences with randomized inter-transaction delays (multi).

cp_error_type — Error Injection Scenario

Purpose: Ensures every category of illegal operation was actually attempted against the DUT during regression — not just defined in the test plan but left unexercised. An assertion that never fires because the illegal stimulus was never driven is not a passing assertion, it's an unexercised one.

Bins:

- status_reg_write = {STATUS_WRITE}
- premature_start = {PRE_START}
- double_start = {DBL_START}

- invalid_dim = {INV_DIM}

Target: All 4 bins ≥ 90%. Each hit by TC-023 through TC-026 respectively.

cp_reset_state — FSM State at Reset

Purpose: Ensures reset was exercised at every FSM state individually — not just from IDLE. A reset recovery bug that only manifests during ACTIVATION_FLOW would never appear if reset tests only happen to hit IDLE or WEIGHT_LOAD.

Bins:

- idle = {IDLE}
- weight_load = {WEIGHT_LOAD}
- pe_clear_state = {PE_CLEAR}
- activation_flow = {ACTIVATION_FLOW}
- done_state = {DONE}

Target: All 5 bins ≥ 90%. Hit by TC-021 (IDLE), TC-018 (WEIGHT_LOAD), TC-019 (PE_CLEAR), TC-020 (ACTIVATION_FLOW), TC-034 (DONE).

7. Scoreboard Architecture & Pass Criteria

The scoreboard has four distinct checking paths. Each one handles a different category of test and has completely separate logic. They must never interfere with each other — a test that enters the wrong path produces either a false pass or a false failure, both of which are silent bugs in the verification infrastructure.

The four paths are:

- Positive Path — normal functional tests, output vs golden model
- Negative Path — error injection tests, no spurious output verification
- Performance Path — latency and throughput verification
- RAL Path — register model correctness, built-in sequence checks

7.1 Path 1 — Positive Checking (Normal Functional Tests)

What It's Doing

This is the primary checking path. It answers the question: did the DUT produce the mathematically correct output for the inputs it was given? Every Category 1, 2, 3, and 4 test uses this path — any test where a legal computation was started and is expected to complete with a valid result.

How the Scoreboard Knows to Use This Path

The scoreboard receives a test type tag from the sequence that started the computation. Every sequence item carries an enum field:

```
typedef enum {     POSITIVE,     NEGATIVE,     PERFORMANCE,     RAL } check_type_e;
```

When the data monitor observes the output buffer after done asserts, it packages the result into a transaction object that includes this tag. The scoreboard dispatches to the correct checking path based on the tag. This is the mechanism that prevents path interference.

The Checking Procedure — Step by Step

Step 1 — Input capture: The data driver sends matrices A and B to the scoreboard via a TLM FIFO at the moment they are driven onto the DUT. The scoreboard stores them in a queue keyed by transaction ID. This happens before the computation completes — the scoreboard is not waiting for the output to arrive before it knows the inputs.

Step 2 — Output capture: The data monitor observes the output buffer after STATUS_REG done asserts. It reads all N×N int32 values, packages them into a result transaction with the matching transaction ID, and sends them to the scoreboard via TLM analysis port.

Step 3 — Golden model call: The scoreboard calls ref_model.py with the stored A and B matrices for this transaction ID via a DPI-C bridge to a Python subprocess (passing matrices as JSON, receiving expected C as JSON). The Python script computes C = np.matmul(A.astype(np.int32), B.astype(np.int32)). The explicit cast to int32 before multiplication is critical — numpy defaults to int64 for safety. The cast enforces the same int8 × int8 → int32 accumulation behavior the RTL implements.

Step 4 — Element-wise comparison: The scoreboard compares every element of the observed output against the golden model result using !== (not !=). In SystemVerilog, != treats X

and Z as wildcards — a result of X would incorrectly pass. !== is the 4-state inequality operator that treats X and Z as mismatches. Any X in the output buffer means an undriven or uninitialized signal reached the output — that is a real bug and must be caught.

Element-wise comparison body:

```
foreach (observed_C[i,j]) begin     if (observed_C[i][j] !== expected_C[i][j]) begin
`uvm_error("SCOREBOARD",             $sformatf("MISMATCH at [%0d][%0d]: expected %0d,
got %0d | Test: %s | Dim: %0d",             i, j, expected_C[i][j], observed_C[i][j],
test_name, dim))     end end
```

Step 5 — Pass declaration: If all N×N elements pass the !== check, the scoreboard increments its pass counter and logs the transaction as clean. At end of simulation, the total pass count is reported.

Pass Criterion

A positive test passes if and only if every element of the N×N DUT output matches the corresponding element of the Python golden model output exactly, with no X or Z values present in the observed output. There is no tolerance. Int32 arithmetic is exact. A result that is off by 1 is a failing result.

What Gets Logged on a Mismatch

Every uvm_error includes the following, as it is the information needed to reproduce the bug, trace it in a waveform, and write a BUGS.md entry:

- Matrix coordinates of the mismatch [i][j]
- Expected value (from golden model)
- Observed value (from DUT output buffer)
- Full input matrices A and B (so the bug is 100% reproducible)
- Transaction ID
- Test name and random seed
- Current FSM state at time of output capture
- Cycle number of done assertion

7.2 Path 2 — Negative Checking (Error Injection Tests)

What It's Doing

This path answers a completely different question: did the DUT correctly do nothing in response to an illegal operation? Every Category 5 test uses this path. There is no golden model call on this path — the expected output is not a matrix, it is the absence of any output at all.

The Checking Procedure — Step by Step

Step 1 — Illegal operation window definition: When an error injection sequence begins, it notifies the scoreboard of two things: the type of illegal operation being driven, and the duration of the observation window (in cycles) during which no output should appear. The window is defined conservatively — long enough that any spurious response the DUT could possibly generate would fall within it.

Step 2 — Spurious output monitor: The scoreboard forks a monitoring thread that watches the output buffer and STATUS_REG done bit for the entire window duration. If done asserts at

any point during the window, it is a scoreboard failure — the DUT responded to an illegal operation.

Spurious output monitor task:

```
task check_no_spurious_output(int unsigned window_cycles, string op_type);     fork
begin : monitor_output             @(posedge done_observed);
`uvm_error("SCOREBOARD",                $sformatf("SPURIOUS OUTPUT during illegal
op: %s at cycle %0d",                op_type, $time))         end         begin :
timeout             repeat(window_cycles) @(posedge clk);             disable
monitor_output;         end     join_any     disable fork; endtask
```

Step 3 — State integrity check: After the illegal operation window closes, the scoreboard reads back all three registers via the RAL model and verifies their values are consistent with the pre-illegal-operation state. A write to STATUS_REG must not have changed its value. A premature START must not have advanced the FSM.

Step 4 — Follow-on clean transaction: Every error injection test ends with a legal computation. This computation uses the positive checking path — the scoreboard verifies its output against the golden model. This is the most important check in the negative path: it proves that the illegal operation left no residual corruption in the DUT's state. A DUT that gracefully ignores the illegal operation but then produces wrong results on the next legal computation has a state corruption bug that only the follow-on check catches.

Pass Criterion

A negative test passes if and only if: (1) no done assertion occurs during the illegal operation window, (2) all register values remain consistent with their pre-illegal-operation state, and (3) the follow-on clean transaction produces a result that exactly matches the Python golden model. All three conditions must hold simultaneously. Passing two out of three is a failing test.

7.3 Path 3 — Performance Checking (Latency and Throughput)

What It's Doing

This path answers: did the DUT produce the correct result in exactly the right number of cycles? Every Category 6 test uses this path. It operates in parallel with the positive checking path — the output is still verified against the golden model, but additionally the cycle count is verified against the 2N contract.

Two Checkers, One Contract

Per the master plan, latency is checked at two different abstraction levels simultaneously. Both must agree.

Samarth's latency_checker (transaction level, inside mmu_scoreboard.sv): Operates on transaction timestamps. Records the cycle number when CTRL_REG start is written. Records the cycle number when STATUS_REG done asserts. Computes observed latency as the difference.

```
if (observed_latency !== (2 * dim)) begin     `uvm_error("LATENCY_CHECKER",
$sformatf("LATENCY VIOLATION: expected %0d cycles, observed %0d | Dim: %0d | Test: %s",
2*dim, observed_latency, dim, test_name)) end
```

Jad's mmu_perf_checker.sv (signal level, bound to mmu_top): Operates on raw RTL signals. The SVA property fires the moment activation_flow_start rises and checks that done asserts exactly 2N cycles later — not one cycle early, not one cycle late.

```
property result_latency;     @(posedge clk) $rose(activation_flow_start)         |->
##[2*N:2*N] $rose(done); endproperty assert property (result_latency);
```

Checker disagreement: If Samarth's transaction-level checker passes but Jad's signal-level SVA fires — or vice versa — this indicates a timing discrepancy between when the RAL model sees the transaction complete and when the RTL signal actually asserts. This is a real bug: either the AXI-Lite slave is reporting done incorrectly, or there is a pipeline register discrepancy. Any disagreement between the two checkers is logged in BUGS.md as a priority-one investigation.

Throughput Check (TC-031)

For back-to-back throughput verification, the scoreboard computes actual_throughput = total_computations / total_cycles against theoretical_max = 1 computation per (2N + weight_load_cycles). Any back-to-back stall cycles not accounted for by the FSM's natural WEIGHT_LOAD phase reduce actual throughput below theoretical maximum and are flagged in perf_report.md with the specific cycle where the stall occurred.

Pass Criterion

A performance test passes if and only if: (1) the DUT output matches the Python golden model exactly (positive check), (2) Samarth's transaction-level latency checker observes exactly 2N cycles between START and DONE, and (3) Jad's signal-level SVA fires no violations. All three must pass. Any disagreement between checkers (2) and (3) is an automatic investigation trigger regardless of whether the functional output is correct.

7.4 Path 4 — RAL Checking (Register Model Correctness)

What It's Doing

This path answers: does the register file behave exactly as the RAL model specifies — correct reset values, correct read-write behavior, correct enforcement of access policies? TC-032 and TC-033 exclusively use this path. No golden model call. No output buffer monitoring. The RAL model itself is the checker.

How the RAL Model Checks Automatically

The RAL model in mmu_reg_model.sv defines every register field with its access type, reset value, and field width. The built-in sequences use these definitions as the expected values:

- uvm_reg_hw_reset_seq reads every register after reset and compares against the defined reset values. The comparison is internal to the RAL framework — no custom scoreboard code needed.
- uvm_reg_access_seq writes a walking-ones pattern to every RW field, reads back after each write, and compares. Any field that doesn't retain its written value is flagged automatically. STATUS_REG is automatically skipped for writes because its access type is "RO" — the RAL model enforces this without custom code.

What the Scoreboard Adds on Top

The RAL built-in sequences handle register correctness. The scoreboard adds one additional check on top: it verifies that running TC-032 and TC-033 did not accidentally trigger a computation. Specifically:

- Done must never assert during either RAL sequence
- The FSM must remain in IDLE throughout
- The output buffer must contain no valid data after either sequence completes

These checks use the same spurious output monitor from the negative path — the RAL sequences are effectively a special case of negative testing from the scoreboard's perspective.

Pass Criterion

A RAL test passes if and only if: (1) the RAL framework's built-in comparisons report zero mismatches for all register fields, (2) no done assertion occurs at any point during the sequence, and (3) the FSM remains in IDLE throughout. TC-032 and TC-033 must both pass before any other test in the regression is considered valid.

7.5 Summary — The Four Pass Criteria

Path	Tests	Pass Condition
Positive	Cat 1, 2, 3, 4	All N×N outputs match golden model exactly, no X/Z
Negative	Cat 5	No spurious done, registers unchanged, follow-on transaction correct
Performance	Cat 6	Output correct + transaction latency == 2N + SVA no violations + checkers agree
RAL	TC-032, TC-033	All register fields correct + no spurious computation triggered

7.6 Global Rule Across All Paths

A single uvm_error anywhere in the simulation — on any path — causes the regression to fail. There is no "mostly passing" in a regression. The Makefile's make regress command exits nonzero if any uvm_error was raised, regardless of how many tests passed before it. This is the industry standard and it is non-negotiable per master plan Rule 15.

8. Assertion Exercise Mapping

The project has three SVA files bound to three different RTL modules, plus one performance checker bound at the top level — nine properties total. Each property is fully mapped below: the RTL signal it guards, the legal stimulus tests expected to exercise it without firing (Mode 1), the directed illegal stimulus designed to make it fire (Mode 2), and the Xcelium activity-report target that confirms it was genuinely active during regression.

File overview:

- axi_lite_sva.sv → bound to axi_lite_slave.sv (2 properties: A1, A2)
- mmu_controller_sva.sv → bound to mmu_controller.sv (4 properties: B1–B4)
- pe_sva.sv → bound to pe.sv (×N×N instances) (1 property: C1)
- mmu_perf_checker.sv → bound to mmu_top.sv (2 properties: D1, D2)

8.1 File 1 — axi_lite_sva.sv (bound to axi_lite_slave.sv)

A1 — axi_awvalid_stable

Once the AXI write address valid signal asserts, it must hold until the slave acknowledges it with awready. A master that drops awvalid before the handshake completes is violating the AXI-Lite protocol — the slave may have missed the transaction entirely.

```
property axi_awvalid_stable;     @(posedge clk) awvalid && !awready |=> awvalid;
endproperty assert property (axi_awvalid_stable);
```

Mode 1 — Should Hold (legal stimulus):

Test	Why It Exercises This
TC-001 through TC-010	Every functional test writes DIM_REG and CTRL_REG via AXI-Lite. The driver holds awvalid until awready — this property checks that behavior is correct on every write transaction.
TC-032, TC-033	RAL sequences generate multiple AXI write transactions — highest density of awvalid/awready handshakes in the regression.

Mode 2 — Should Fire: Add to TC-023's error injection sequence: after asserting awvalid for the STATUS_REG write attempt, drop awvalid one cycle before awready would assert. This is the stimulus that proves A1 is actually watching.

Xcelium Verification: A1 triggered on every AXI write transaction — minimum once per functional test plus the deliberate violation in TC-023.

A2 — axi_wvalid_with_awvalid

In AXI-Lite, write address and write data are sent simultaneously — there is no separate data channel timing like in full AXI4. If awvalid asserts without wvalid, the slave has a write address but no data — undefined behavior. This property enforces the simultaneous assertion requirement.

```
property axi_wvalid_with_awvalid;     @(posedge clk) awvalid |-> wvalid; endproperty
assert property (axi_wvalid_with_awvalid);
```

Mode 1 — Should Hold (legal stimulus):

Test	Why It Exercises This
TC-001 through TC-010	Every DIM_REG and CTRL_REG write drives awvalid and wvalid simultaneously — legal AXI-Lite behavior.
TC-032, TC-033	High-density register write traffic — multiple simultaneous assertions per sequence.

Mode 2 — Should Fire: Add to TC-025's double-START sequence: the second START write attempt drives awvalid without wvalid to test the slave's handling of a malformed write. This proves A2 fires when the protocol is actually violated.

Xcelium Verification: A2 must trigger on every awvalid assertion in the simulation. Zero triggers means the AXI driver never asserted awvalid — a fundamental driver bug.

8.2 File 2 — mmu_controller_sva.sv (bound to mmu_controller.sv)

B1 — no_skip_weight_load

The FSM must always enter WEIGHT_LOAD as the first state after START — it cannot jump directly to PE_CLEAR or ACTIVATION_FLOW. A controller bug that skips weight loading entirely would produce outputs based on whatever garbage values happen to be in the PE registers — a silent catastrophic failure.

```
property no_skip_weight_load;     @(posedge clk) (state == IDLE) && start |=> (state == WEIGHT_LOAD); endproperty assert property (no_skip_weight_load);
```

Mode 1 — Should Hold (legal stimulus):

Test	Why It Exercises This
TC-001 through TC-013	Every functional test asserts START from IDLE — B1 checks the transition on every single computation.
TC-022	Reset then immediate restart — especially important since the FSM just returned from reset to IDLE.
TC-034	Post-DONE reset then restart — another IDLE → START transition after an unusual path.

Mode 2 — Should Fire: TC-035a: force state = PE_CLEAR while in IDLE with start asserted. B1 must fire immediately. This is a low-level directed test that doesn't need to be a full UVM sequence — a simple directed test in the integration phase suffices.

Xcelium Verification: B1 must trigger on every START assertion in the simulation — the trigger count must equal the total number of computations started across the entire regression.

B2 — pe_clear_one_cycle

pe_clear must assert for exactly one cycle. If it holds for two or more cycles, the controller's PE_CLEAR state is longer than spec — wasting cycles and violating the 2N latency contract. If it pulses twice, something is deeply wrong with the FSM. This is one of the highest-value assertions in the project because a pe_clear timing bug causes both latency violations and potential accumulator corruption.

```
property pe_clear_one_cycle;     @(posedge clk) $rose(pe_clear) |=> !pe_clear;
endproperty assert property (pe_clear_one_cycle);
```

Mode 1 — Should Hold (legal stimulus):

Test	Why It Exercises This
TC-001 through TC-013	Every computation triggers exactly one pe_clear pulse — B2 checks the duration on every single one.
TC-011, TC-012, TC-013	Back-to-back sequences — B2 must hold across every transaction in a 10+ computation chain.
TC-018, TC-019, TC-020	Reset stress sequences — pe_clear must still be exactly one cycle on the post-reset computation.

Mode 2 — Should Fire: TC-035b: force state = PE_CLEAR for two consecutive cycles with pe_clear asserted. B2 must fire on the second cycle when pe_clear fails to deassert.

Xcelium Verification: B2 must trigger once per computation — trigger count must equal total computation count across regression, same as B1.

B3 — result_latency

Done must assert exactly 2N cycles after activation flow begins — not 2N−1, not 2N+1. This is the pipelined latency contract locked in the master plan. It is checked here at the signal level by the SVA and simultaneously at the transaction level by Samarth's latency_checker — both must agree per Section 7 Path 3.

```
property result_latency;     @(posedge clk) $rose(activation_flow_start)         |->
##[2*N:2*N] $rose(done); endproperty assert property (result_latency);
```

Mode 1 — Should Hold (legal stimulus):

Test	Why It Exercises This
TC-027	N=1, 2-cycle latency — tightest case, most sensitive to off-by-one.
TC-028	N=2, 4-cycle latency.
TC-029	N=3, 6-cycle latency.
TC-030	N=4, 8-cycle latency — worst case.
TC-011, TC-031	Back-to-back sequences — B3 must hold on every computation in the chain.

Mode 2 — Should Fire: TC-035c: force done = 1 at cycle 2N−1 after activation_flow_start — B3 must fire. TC-035d: suppress done until cycle 2N+1 — B3 must fire again. These are the two failure modes the property is guarding against.

Xcelium Verification: B3 must trigger once per computation. For a regression with 500+ simulation runs, confirm the trigger count matches total computation count.

B4 — no_spurious_done

Done must never assert while the FSM is in IDLE. A spurious done assertion from IDLE could cause the testbench to read garbage from the output buffer, or trigger a false pass in a poorly

written scoreboard. This catches a class of RTL bugs where the done signal is combinatorially derived incorrectly and glitches high briefly during state transitions.

```
property no_spurious_done;     @(posedge clk) (state == IDLE) |-> !done; endproperty
assert property (no_spurious_done);
```

Mode 1 — Should Hold (legal stimulus):

Test	Why It Exercises This
TC-021, TC-022	Reset tests — FSM in IDLE after reset, done must stay low.
TC-032, TC-033	RAL sequences — FSM stays in IDLE throughout, done must never assert.
TC-024	Premature START with dim=0 — FSM stays in IDLE, done must never assert.
All Category 1 tests	Between transactions the FSM returns to IDLE — B4 checks done is low during every inter-transaction IDLE period.

Mode 2 — Should Fire: TC-035e: force done = 1 while state == IDLE. B4 must fire immediately on the cycle where both conditions are true simultaneously. This is the cleanest and fastest forced violation to implement.

Xcelium Verification: B4 is checked every cycle the FSM spends in IDLE — this is the highest trigger-count assertion in the project. Across a full regression it should trigger thousands of times. A low trigger count means the FSM rarely visits IDLE — investigate immediately.

8.3 File 3 — pe_sva.sv (bound to pe.sv, all N×N instances)

C1 — zero_input_no_accumulate

When the activation flowing into a PE is zero, the accumulator must not change — because 0 × weight = 0 regardless of weight value, so accumulation of zero adds nothing. If the accumulator changes when activation_in is zero, either the MAC logic is broken or there is a spurious clock enable issue.

Important: this assertion is bound to every PE instance — in a 4×4 array that is 16 simultaneous copies of this property running in parallel. The assertion activity report will show 16× the trigger count of a single-PE assertion.

```
property zero_input_no_accumulate;     @(posedge clk) (activation_in == 0) |-> (acc ==
$past(acc)); endproperty assert property (zero_input_no_accumulate);
```

Mode 1 — Should Hold (legal stimulus):

Test	Why It Exercises This
TC-005	All-zero activation matrix — every PE receives zero every cycle; C1 holds for all 16 instances every cycle of ACTIVATION_FLOW.
TC-010	Identity weight matrix with random activations — PEs on the off-diagonal receive zero activations for specific cycles.
TC-014	Weight poison all-zero — zero weight × any activation = 0 product, accumulator change is zero.

Mode 2 — Should Fire: TC-035f: during a zero-activation cycle, force a specific PE's accumulator (e.g. PE[1][2]) to increment by 1. C1 must fire on that PE instance. This is the most surgical forced violation in the project — it targets a specific PE instance at a specific cycle.

Xcelium Verification: C1 must trigger on every cycle where any PE receives a zero activation. In TC-005 (all-zero activations, 4×4, 8 cycles of ACTIVATION_FLOW) this is 16 PEs × 8 cycles = 128 triggers from that test alone. Confirm the trigger count is consistent with the number of zero-activation cycles driven across the regression.

8.4 File 4 — mmu_perf_checker.sv (bound to mmu_top.sv)

D1 — signal_level_latency

This is Jad's signal-level counterpart to B3. B3 is bound to mmu_controller.sv and checks internal controller signals. D1 is bound to mmu_top.sv and checks the top-level interface signals — the done that the testbench actually sees. The distinction matters: a pipeline register bug between the controller's internal done and the top-level done output would pass B3 but fail D1. Having both is the deliberate redundancy.

```
property signal_level_latency;     @(posedge clk) $rose(activation_flow_start)
|-> ##[2*N:2*N] $rose(done); endproperty assert property (signal_level_latency);
```

Mode 1 — Should Hold (legal stimulus):

Test	Why It Exercises This
TC-027 through TC-030	All four latency verification tests — primary exercise of D1 at every dimension.
TC-031	Back-to-back throughput — D1 must hold on every computation in the chain at the top-level signal.

Mode 2 — Should Fire: TC-035c and TC-035d applied at the mmu_top boundary. Both B3 and D1 should fire simultaneously. If only one fires, there is a signal path discrepancy between the controller and the top level — which is itself a bug worth logging.

Xcelium Verification: D1 triggers once per computation. Trigger count must match B3 exactly.

D2 — back_to_back_throughput

In a back-to-back sequence, after done asserts and a new START is issued, the next activation flow must begin within exactly weight_load_cycles cycles — the time needed to preload the new weights. Any additional stall cycles beyond WEIGHT_LOAD are a throughput bug. This is the signal-level enforcement of the theoretical maximum throughput claim.

```
property back_to_back_throughput;     @(posedge clk) $rose(done) && start_pending
|-> ##[weight_load_cycles:weight_load_cycles] $rose(activation_flow_start); endproperty
assert property (back_to_back_throughput);
```

Mode 1 — Should Hold (legal stimulus):

Test	Why It Exercises This
TC-011	10+ back-to-back N=4 computations, no gap — primary exercise of D2.
TC-031	Dedicated throughput test — D2 must hold on every inter-transaction boundary.

Test	Why It Exercises This
TC-013	Alternating dimensions — D2 must hold even when WEIGHT_LOAD duration changes between transactions.

Mode 2 — Should Fire: TC-035g: force the FSM to remain in WEIGHT_LOAD for one extra cycle beyond spec. D2 must fire on the cycle where activation_flow_start is late.

Xcelium Verification: D2 triggers once per back-to-back transaction boundary — trigger count must equal (total back-to-back computations − 1) across the regression, since the first computation in any sequence has no prior done to trigger from.

8.5 TC-035 — Directed Forced Violation Test Summary

TC-035 uses SystemVerilog force and release statements to deliberately create illegal RTL conditions — signals forced to values the design would never produce under legal stimulus. Its sole purpose is to prove that every Mode 2 assertion fires when it should. The scoreboard uses the negative checking path for all sub-tests: no golden model call, no output comparison. The pass criterion for TC-035 is that every listed assertion fires exactly once per sub-test. A sub-test where the targeted assertion does not fire is a failing test — it means the assertion is broken, not the design.

Sub-test	Signal Forced	Assertion That Must Fire
TC-035a	FSM state forced IDLE→PE_CLEAR, skipping WEIGHT_LOAD	B1
TC-035b	pe_clear held for 2 cycles	B2
TC-035c	done forced at cycle 2N−1 (one cycle early)	B3, D1
TC-035d	done forced at cycle 2N+1 (one cycle late)	B3, D1
TC-035e	done forced while state==IDLE	B4
TC-035f	PE accumulator forced to increment on zero activation	C1
TC-035g	Extra WEIGHT_LOAD stall cycle in back-to-back sequence	D2

8.6 Summary — All Nine Assertions

ID	Assertion	File	Mode 1 Tests	Mode 2 Test	Trigger Frequency
A1	axi_awvalid_stable	axi_lite_sva.sv	TC-001 to TC-010, TC-032, TC-033	TC-023 + TC-035	Once per AXI write
A2	axi_wvalid_with_awvalid	axi_lite_sva.sv	TC-001 to TC-010, TC-032, TC-033	TC-025 + TC-035	Once per AXI write

ID	Assertion	File	Mode 1 Tests	Mode 2 Test	Trigger Frequency
B1	no_skip_weight_load	mmu_controller_sva.sv	TC-001 to TC-013, TC-022, TC-034	TC-035a	Once per START
B2	pe_clear_one_cycle	mmu_controller_sva.sv	TC-001 to TC-013, TC-018 to TC-020	TC-035b	Once per computation
B3	result_latency	mmu_controller_sva.sv	TC-027 to TC-031	TC-035c, TC-035d	Once per computation
B4	no_spurious_done	mmu_controller_sva.sv	TC-021, TC-022, TC-024, TC-032, TC-033	TC-035e	Every IDLE cycle
C1	zero_input_no_accumulate	pe_sva.sv	TC-005, TC-010, TC-014	TC-035f	Every zero-activation cycle × 16 PEs
D1	signal_level_latency	mmu_perf_checker.sv	TC-027 to TC-031	TC-035c, TC-035d	Once per computation
D2	back_to_back_throughput	mmu_perf_checker.sv	TC-011, TC-013, TC-031	TC-035g	Once per back-to-back boundary